

Dashboard de Execução

Rodado agora — 2026-09-08 (horário local). Suíte de testes, stack Docker (Postgres+pgvector e Ollama já com os modelos baixados) e a API exercitada com cenários reais, incluindo isolamento entre tenants, autenticação por chave de API e rate limit.

●

Tudo conforme o esperado: 65/65 testes, 2/2 serviços de infra saudáveis, 10/10 cenários de API com o resultado previsto — inclusive o corte exato do rate limit por (IP, tenant).

65/65

testes passando

97%

cobertura total

2/2

serviços de infra saudáveis

10/10

cenários de API OK

Testes

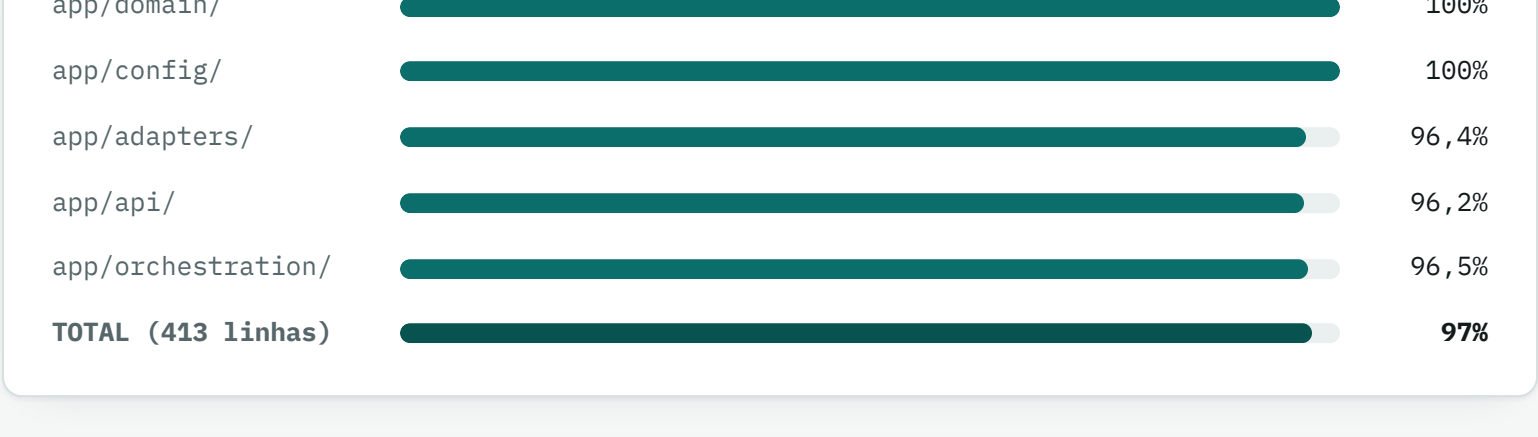
python -m pytest, nas variações do manual — sem Docker para unit/feature;

tests/evals exige o Postgres+pgvector e o Ollama reais, já de pé.

COMANDO	RESULTADO	TEMPO
pytest	● 65 passed	7,98s
pytest --cov=app	● 65 passed ● 97% cov	7,49s
pytest tests/unit	● 54 passed	5,03s
pytest tests/feature	● 11 passed	4,84s
pytest tests/evals -v	● 1 passed ● recall@3 = 1,00	5,63s
ruff check .	● sem apontamentos	<1s

Cobertura por camada

Domínio e config em 100% (regra pura, fácil de isolar). orchestration/ segue a mais baixa: model_gateway.py tem 3 linhas não cobertas (94%) — o except best-effort de litellm.completion_cost, não exercitado pela suíte mockada.



Stack Docker + API local

docker compose up -d postgres ollama — os containers e os modelos (qwen2.5:3b, all-minilm) já existiam de uma sessão anterior, sem novo download. A API rodou fora do container via uvicorn --port 8001, como o próprio README recomenda para dev.

SERVIÇO	IMAGEM	STATUS	PORTA
postgres	pgvector/pgvector:pg16	● healthy	5432
ollama	ollama/ollama:latest	● up	11434
app (uvicorn local)	.venv, fora do container	● up	8001

Cenários de API

Exercitados via curl direto contra a stack acima — sem mock, com os dados de exemplo já ingeridos (4 documentos + pedidos por tenant).

200

GET /health e GET /ready
Liveness sem checar nada externo; readiness abre uma conexão asyncpg real e confirma o Postgres acessível.

ready: {"status": "ready"}

401

POST /chat sem Authorization
Rejeitado antes de tocar no grafo ou no modelo — obter_tenant_id nem chega a consultar o banco.

{"detail": "Header Authorization: Bearer <chave de API> é obrigatório"}

401

POST /chat · Authorization: Bearer chave-fake
Hash da chave não bate com nenhuma linha de tenants — rejeitado do mesmo jeito que a ausência do header.

{"detail": "Chave de API inválida"}

200

POST /chat · Authorization: Bearer dev-loja-azul
Chave real, semeada por seed_tenants.py — hash bate contra tenants.chave_api_hash, tenant resolvido corretamente, isolamento (abaixo) confirmado com a nova autenticação.

resposta: "Pedido 1001: 0 status atual é 'em transporte' e a previsão de entrega é para 2026-09-02."

422

POST /chat · mensagem = " " (só espaços)
Validação Pydantic rejeita antes de qualquer I/O.

{"detail":[{"msg":"Value error, mensagem não pode ser vazia", ...}]}

200

RAG · loja-azul "Qual o prazo para devolução de um produto?"
buscar_documentos recupera o documento certo e a resposta bate com a política real da loja.

resposta (streaming, remontada): "O prazo para devolução de um produto na Loja Azul é de até 7 dias corridos após o recebimento do produto (...). O reembolso é feito no mesmo meio de pagamento usado na compra, em até 10 dias úteis após a confirmação do recebimento pela loja."

200

Tool calling · loja-azul "Qual o status do pedido 1001?"
consultar_pedido bate exatamente com o dado seedado (status "em transporte", previsão 2026-09-02).

resposta: "Seu pedido 1001 está em transporte. A previsão de entrega é até 02/09/2026."

200

Isolamento entre tenants · loja-verde (chave dev-loja-verde) pergunta pelo pedido 1001 (que é da loja-azul)
Mesmo número de pedido, tenant errado — a query no Postgres nem enxerga a linha da outra loja.

resposta: "Desculpe, o número do seu pedido 1001 não foi encontrado. Você pode fornecer outro número de pedido ou compartilhar mais detalhes..."

200

Guardrail de PII · loja-azul "Meu email é cliente.teste@exemplo.com, pode confirmar o pedido 1002?"
O e-mail é mascarado no log da interação antes de ser persistido — a resposta ao cliente segue correta.

resposta: "O seu pedido 1002 já foi entregue. A previsão de entrega foi 20 de agosto de 2026."
log real (mensagem_usuario): "Meu email e [E-MAIL OCULTADO], pode confirmar o pedido 1002?"

200

/

429

Rate limit · 23 POST /chat concorrentes, mesmo IP, dentro de 1 minuto
Corte exato no limite configurado — ver seção dedicada abaixo. Cota é por (IP, tenant) desde esta sessão — um segundo tenant no mesmo IP não herda o bloqueio.

Rate limit em ação

23 POST /chat disparados concorrentemente do mesmo IP e mesmo tenant (limite: 20/minute por par IP+tenant — ADR-012). As 20 primeiras processam normalmente; as 3 seguintes recebem 429 em menos de 1s, sem gastar uma única chamada de modelo — a checagem do slowapi acontece antes do grafo rodar. Um segundo tenant no mesmo IP não compartilha essa cota (testado separadamente, mockado, sem custo de LLM) — ver Arquitetura.

● 200 aceito

● 429 bloqueado

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

Corpo real do bloqueio: {"error": "Rate limit exceeded: 20 per 1 minute"}. As 20 aceitas competem pelo mesmo Ollama local (CPU): a chamada de modelo mais rápida da rajada levou 6,7s (sem fila), a mais lenta 96,4s (23ª a ser atendida) — média de 67s sob essa carga deliberada. Uma chamada isolada, fora de rajada, fica na faixa de 25-45s por etapa (tool-decision + resposta final), compatível com a estimativa de 60-90s do README.

Latência por conversa

A resposta final é sempre a última rodada do loop de ferramentas (ADR-006) quando o modelo fecha a decisão dentro do limite; quando não fecha, o corte de MAX_ITERACOES entra e uma última chamada em texto puro resolve (ADR-006, ADR-005). Duas conversas isoladas (sem concorrência com outras requisições), ambas esgotando as 4 rodadas antes da resposta final:

CONVERSA	CHAMADAS DE MODELO	TEMPO TOTAL
"Oi"	6 (5 decisão + 1 resposta final)	~198s
"Qual o status do pedido 1001?"	6 (5 decisão + 1 resposta final)	~53s

A pergunta sobre o pedido fecha com o dado real ("em transporte", previsão 2026-09-02) — o corte de segurança resolve a partir do que já foi apurado nas rodadas anteriores, mesmo sem o modelo ter sinalizado "não preciso de mais ferramenta". Em ambos os casos a resposta final chega como texto corrido, em um único bloco SSE — nunca como sintaxe de tool call solta.

Observabilidade — logging JSON estruturado

Cada chamada de modelo e cada interação completa saem como uma linha JSON no stdout — sem Langfuse, sem UI externa (ADR-010).

CHAMADA_MODELO CONCLUÍDA (REAL)

"tenant_id": "loja-azul",

"prompt_id": "chat.agente@v1",

"model": "ollama_chat/qwen2.5:3b",

"latencia_segundos": 26.712,

"custo_usd": 0.0,

"tokens_entrada": 413,

"tokens_saida": 23

INTERACAO_CHAT CONCLUÍDA (REAL, PII MASCARADO)

"tenant_id": "loja-azul",

"prompt_id": "chat.agente@v1",

"mensagem_usuario": "Meu email e [E-MAIL OCULTADO], pode confirmar o pedido 1002?",

"resposta": "O seu pedido 1002 já foi entregue..."

⚠ **Encoding do log no Windows, fora do Docker.** Rodando o uvicorn local neste host (não containerizado), o log em stdout sai no codepage do console (cp1252) em vez de UTF-8 — "concluída" vira bytes inválidos como UTF-8 (conclu\xE0da) ao ser redirecionado pra arquivo, sem PYTHONIOENCODING=utf-8. A regra de mascaramento em si não é afetada (o e-mail sai ocultado corretamente); o problema é só de transporte do log nesse console específico. Dentro do container Docker (Linux) o encoding padrão já é UTF-8.

Stack deixado no ar de propósito — postgres e ollama saudáveis, API local em http://localhost:8001 — pra você continuar explorando pelo Swagger em http://localhost:8001/docs. Quando terminar, docker compose down encerra a infra e Ctrl+C encerra o uvicorn.